



Quiz 2

- ▶ 15 minutes
- ▶ Please write your name and EID on all pages
- ▶ Don't turn over your paper until the quiz starts

CS388: Natural Language Processing

Lecture 11: Understanding In- Context Learning



Slide credits: Greg Durrett



Administration

- ▶ Project proposals: form groups of 2 if you like, proposal due 23rd.



This Lecture

- ▶ Prompting: best practices and why it works
 - ▶ Zero-shot prompting: role of the prompt
 - ▶ Few-shot prompting (in-context learning): characterizing demonstrations
- ▶ Understanding in-context learning
 - ▶ ICL can learn linear regression
 - ▶ Induction heads and mechanistic interpretability

Zero-shot Prompting



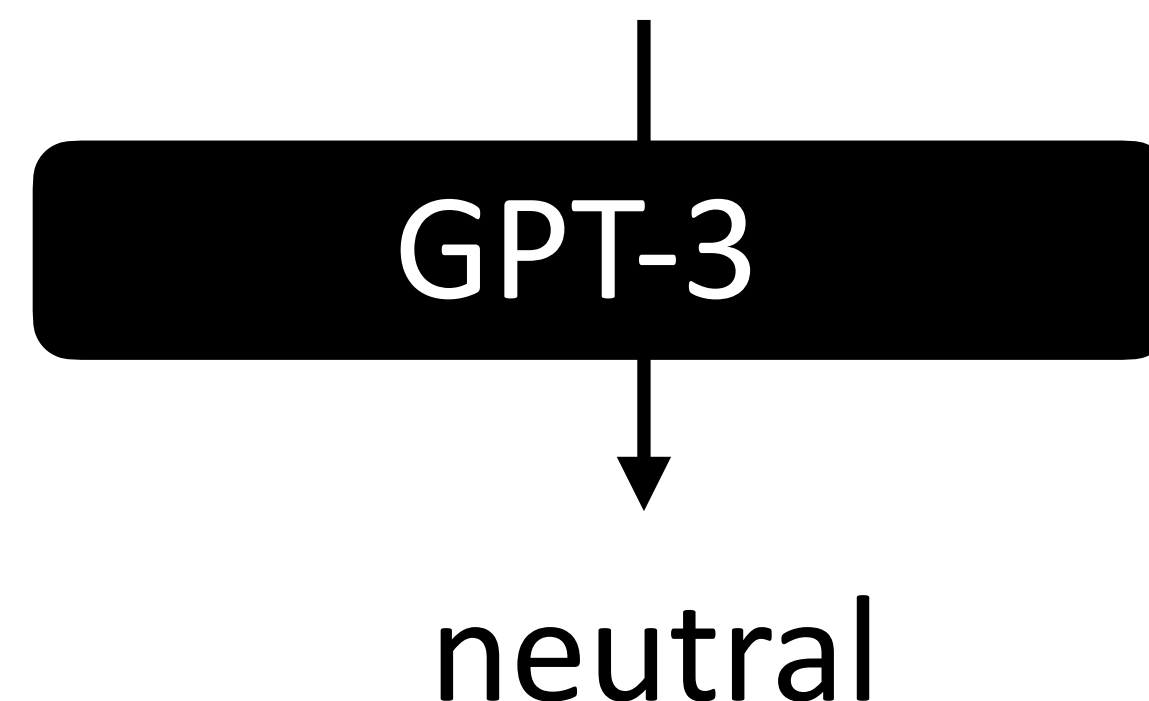
Zero-shot Prompting

- ▶ Single unlabeled datapoint $\mathbf{x}$, want to predict label y
 $\mathbf{x} =$ *The movie's acting could've been better, but the visuals and directing were top-notch.*

- ▶ Wrap $\mathbf{x}$ in a template we call a **verbalizer** $\mathbf{v}$

Review: The movie's acting could've been better, but the visuals and directing were top-notch.

Out of positive, negative, or neutral, this review is





Zero-shot Prompting

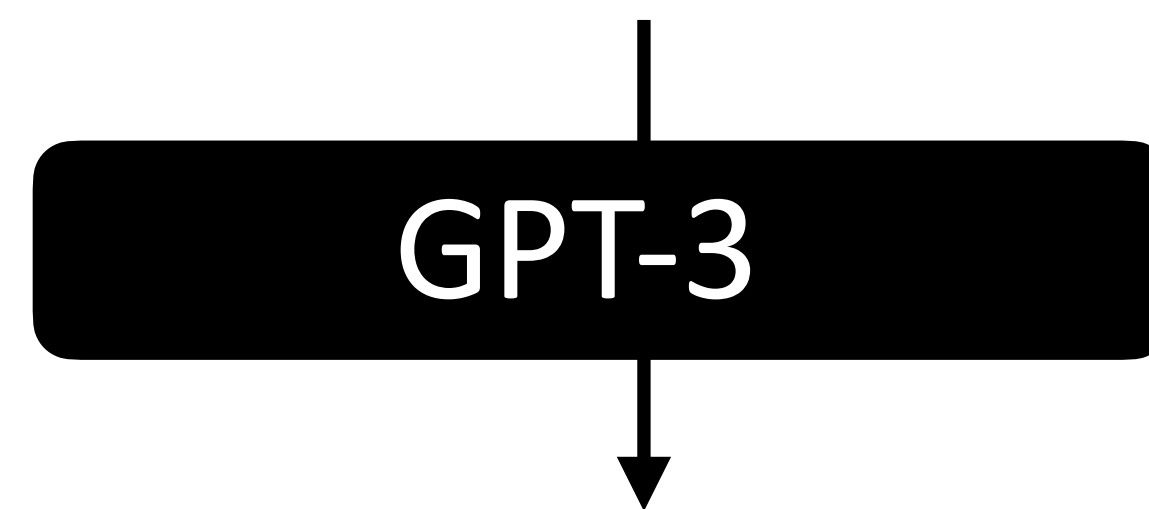
- ▶ Single unlabeled datapoint $\mathbf{x}$, want to predict label y

$\mathbf{x}$ = *The movie's acting could've been better, but the visuals and directing were top-notch.*

- ▶ Wrap $\mathbf{x}$ in a template we call a **verbalizer** $\mathbf{v}$

Review: *The movie's acting could've been better, but the visuals and directing were top-notch.*

On a 1 to 4 star scale, the reviewer would probably give this movie



3 stars.



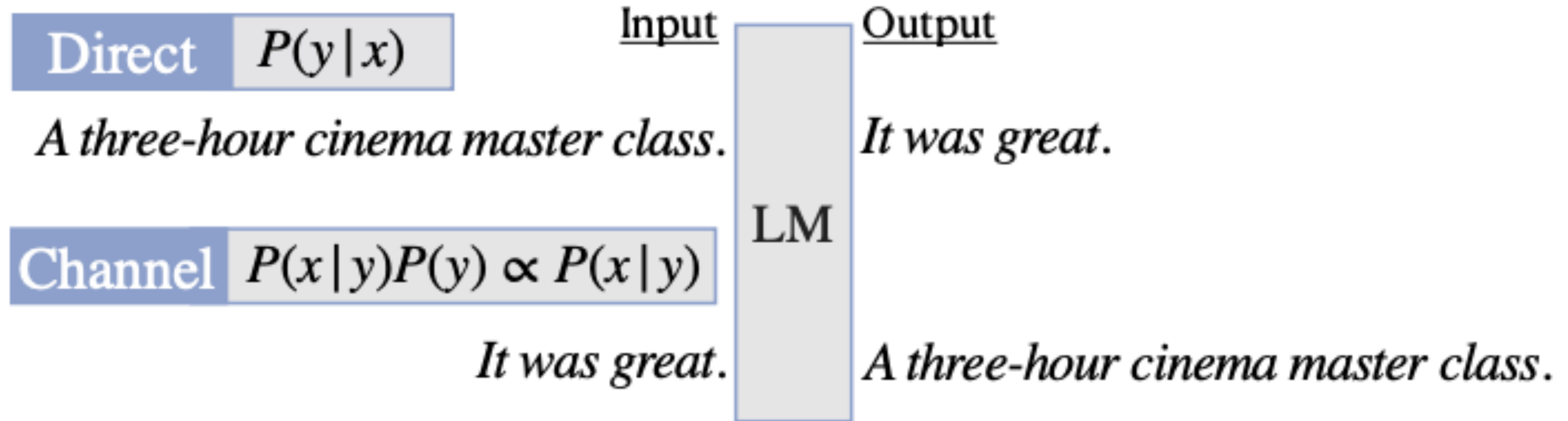
Ways to do classification

- ▶ Generate from the model and read off the generation
 - ▶ What if you ask for a star rating and it doesn't give you a number of stars but just says something else?
- ▶ Compare probs: "*Out of positive, negative, or neutral, this review is _*"
Compare $P(\text{positive} \mid \text{context})$, $P(\text{neutral} \mid \text{context})$, $P(\text{negative} \mid \text{context})$
 - ▶ This constrains the model to only output a valid answer, and you can normalize these probabilities to get a distribution



Ways to do classification

$(x, y) = (\text{"A three-hour cinema master class."}, \text{"It was great."})$

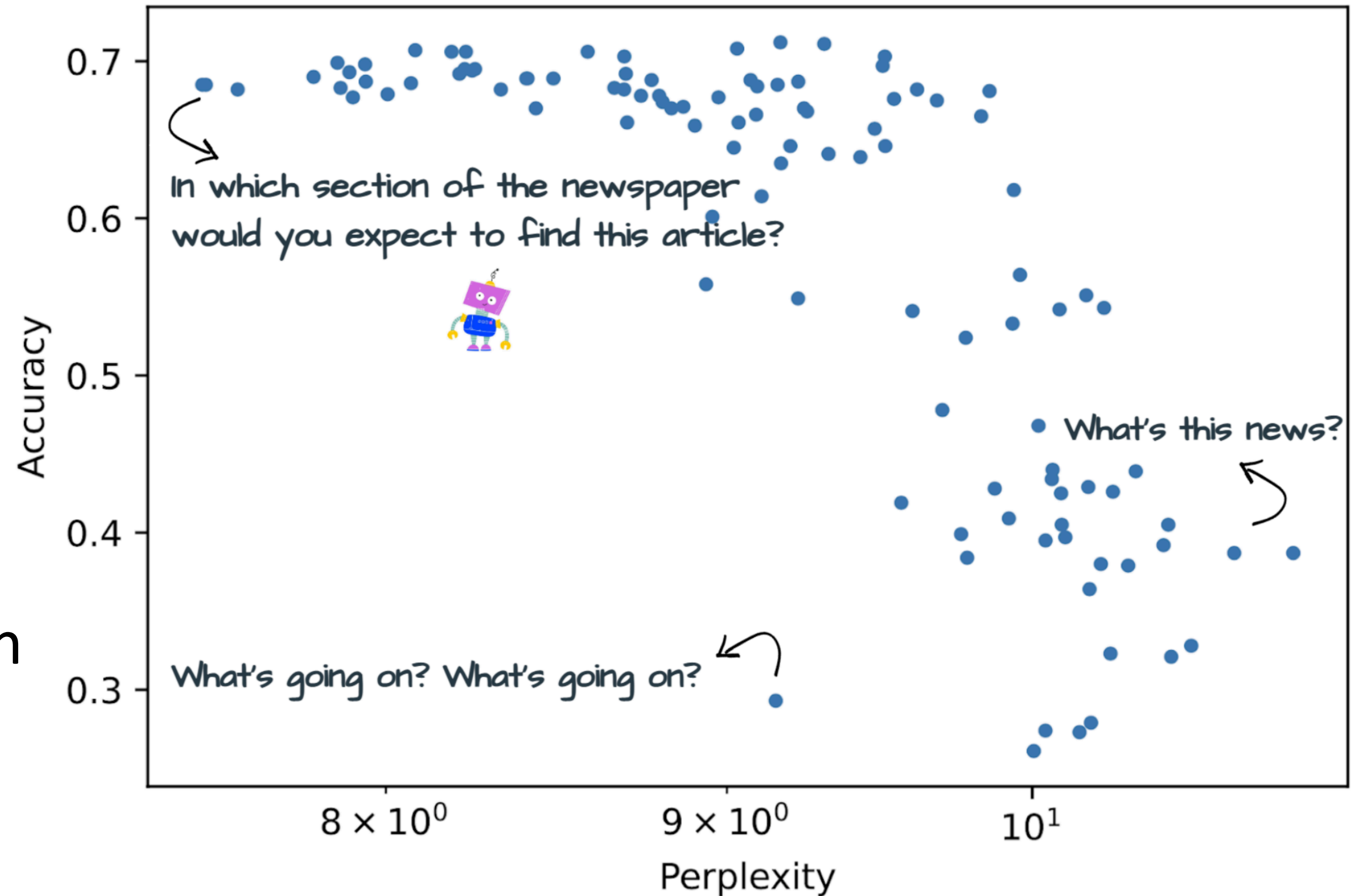


- ▶ Can also compute probabilities of **examples** given **labels** (“noisy channel” method)



Variability in Prompts

- ▶ Plot: large number of prompts produced by {manual writing, paraphrasing, backtranslation}
- ▶ x-axis: perplexity of the prompt. How natural is it? How much does it appear in the pre-training data?
- ▶ y-axis: task performance





Variability in Prompts

- ▶ OPT-175B: average of best 50% of prompts is much better than average over all prompts

Task	Avg Acc	Acc 50%
Antonyms	—	—
GLUE Cola	47.7	57.1
Newspop	66.4	72.9
AG News	57.5	68.7
IMDB	86.2	91.0
DBpedia	46.7	55.2
Emotion	16.4	23.0
Tweet Offensive	51.3	55.8



Prompt Optimization

- ▶ A number of methods exist for searching over prompts (either using gradients or black-box optimization)
- ▶ These can lead to better prompts under the right circumstances (Agrawal et al. (2025))
- ▶ Nevertheless, the choice of prompt *is* very important for zero-shot settings! We will see more next time.
- ▶ In two lectures: models that are trained to do better at prompts (RLHF)

Few-shot Prompting



Few-shot Prompting

- ▶ Form “training examples” from $(\mathbf{x}, y)$ pairs, verbalize them (can be lighter-weight than zero-shot verbalizer)
- ▶ Input to GPT-3: $\mathbf{v}(\mathbf{x}_1) \mathbf{v}(y_1) \mathbf{v}(\mathbf{x}_2) \mathbf{v}(y_2) \dots \mathbf{v}(\mathbf{x}_{\text{test}})$

Review: The cinematography was stellar; great movie!

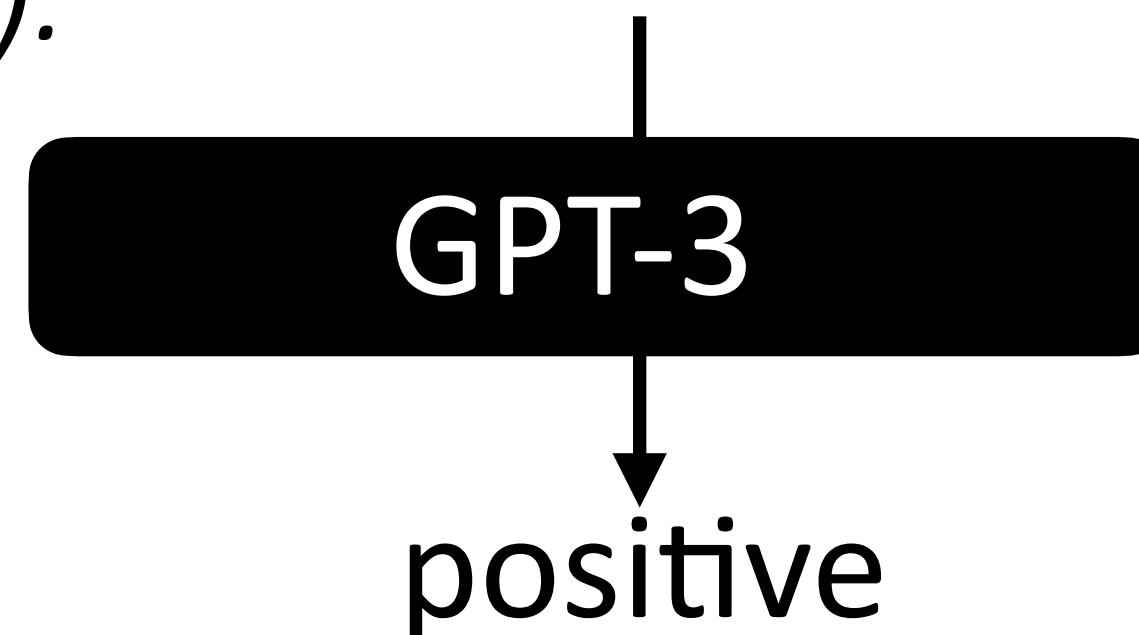
Sentiment (positive or negative): positive

Review: The plot was boring and the visuals were subpar.

Sentiment (positive or negative): negative

Review: The movie's acting could've been better, but the visuals and directing were top-notch.

Sentiment (positive or negative):





What can go wrong?

Review: The movie was great!

Sentiment: positive

Review: I thought the movie was alright; I would've seen it again.

Sentiment: positive

Review: The movie was pretty cool!

Sentiment: positive

Review: Pretty decent movie!

Sentiment: positive

Review: The movie had good enough acting and the visuals were nice.

Sentiment: positive

Review: There wasn't anything the movie could've done better.

Sentiment: positive

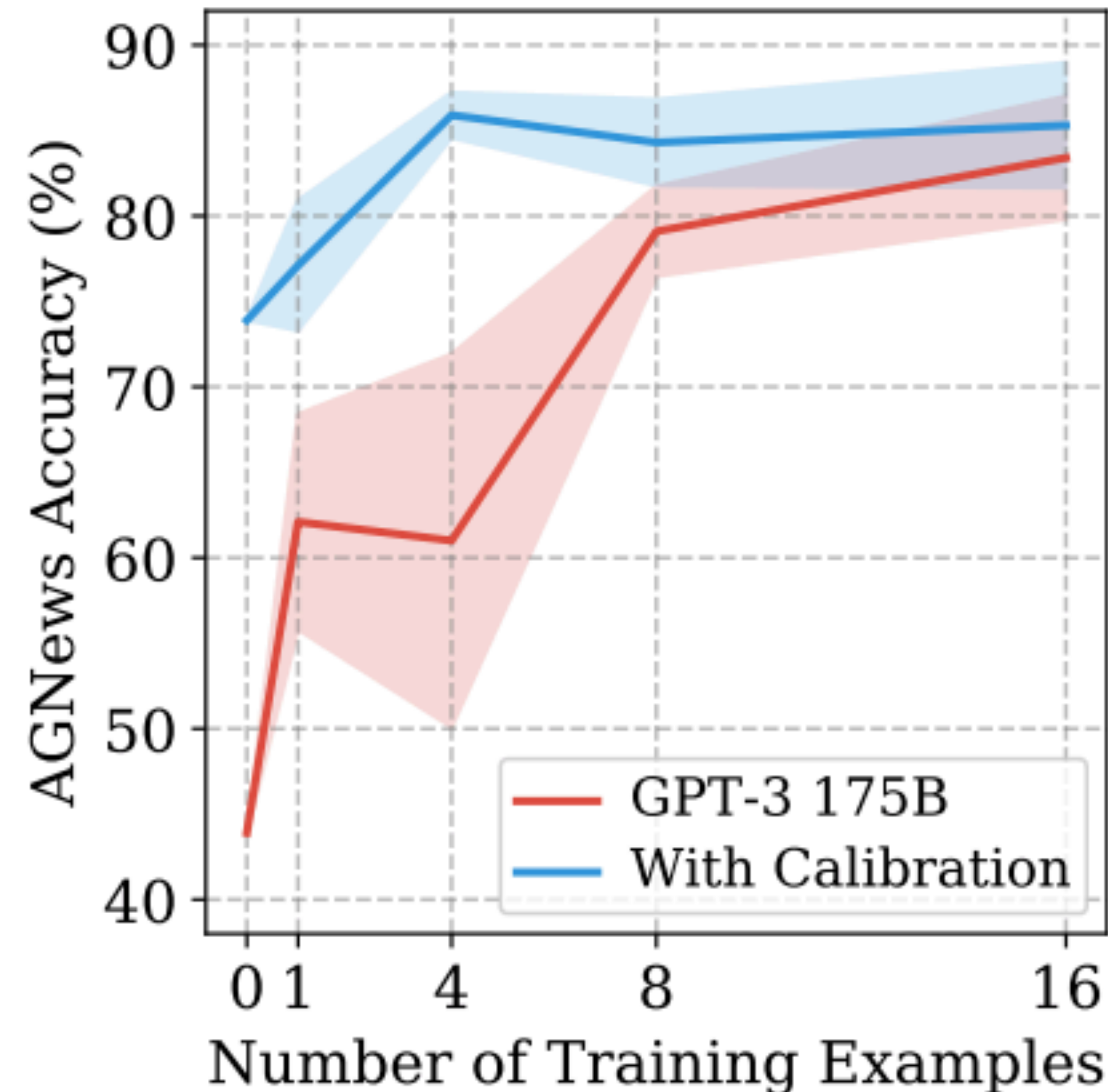
Review: Okay movie but could've been better.

Sentiment: — **GPT-3** → positive



What can go wrong?

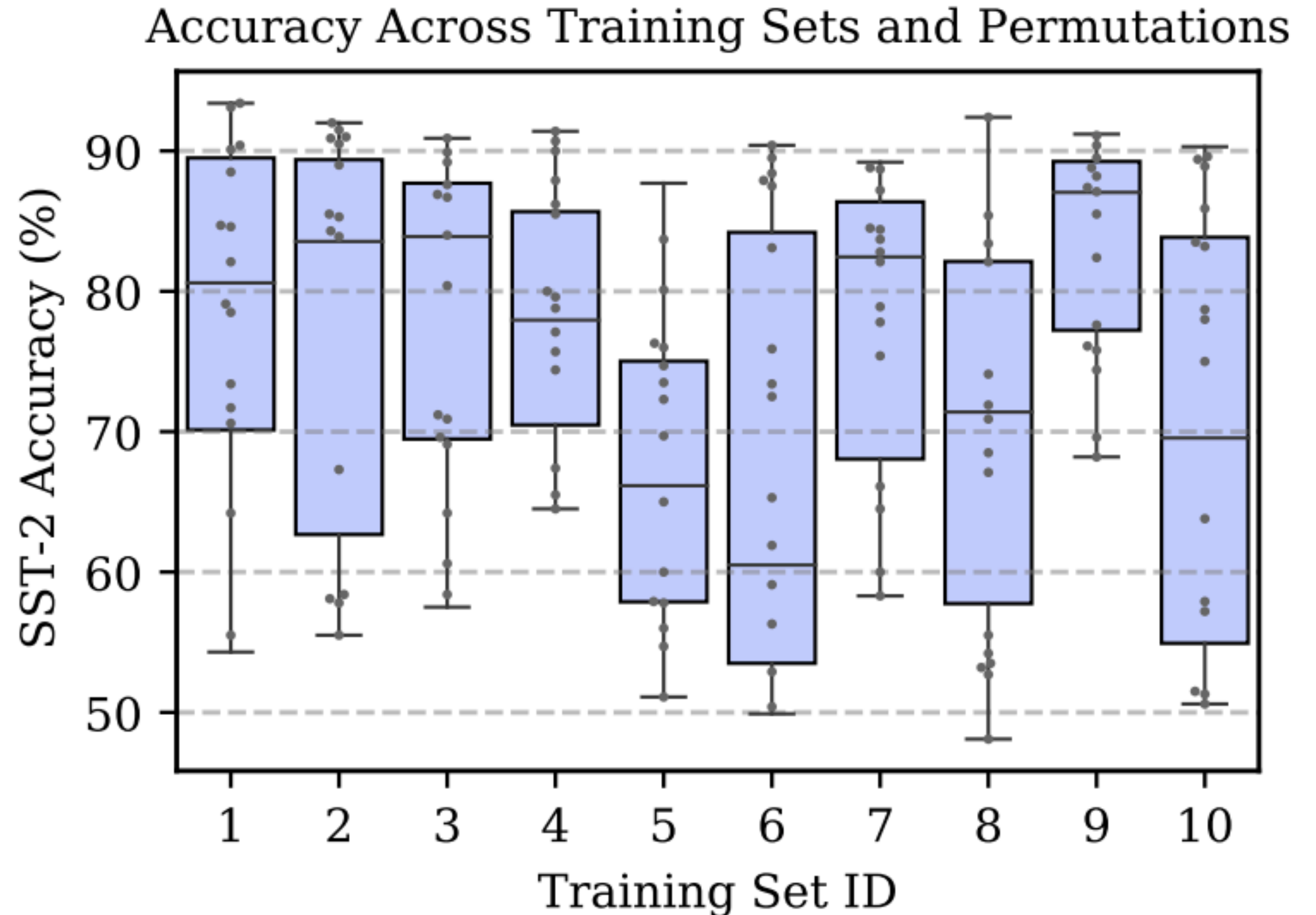
- ▶ All one training label — model sees extremely skewed distribution
- ▶ What if we take random sets of training examples? There is quite a bit of variance on basic classification tasks
- ▶ Note: these results are with basic GPT-3 and not Instruct-tuned versions of the model. This issue has gotten a lot better





What can go wrong?

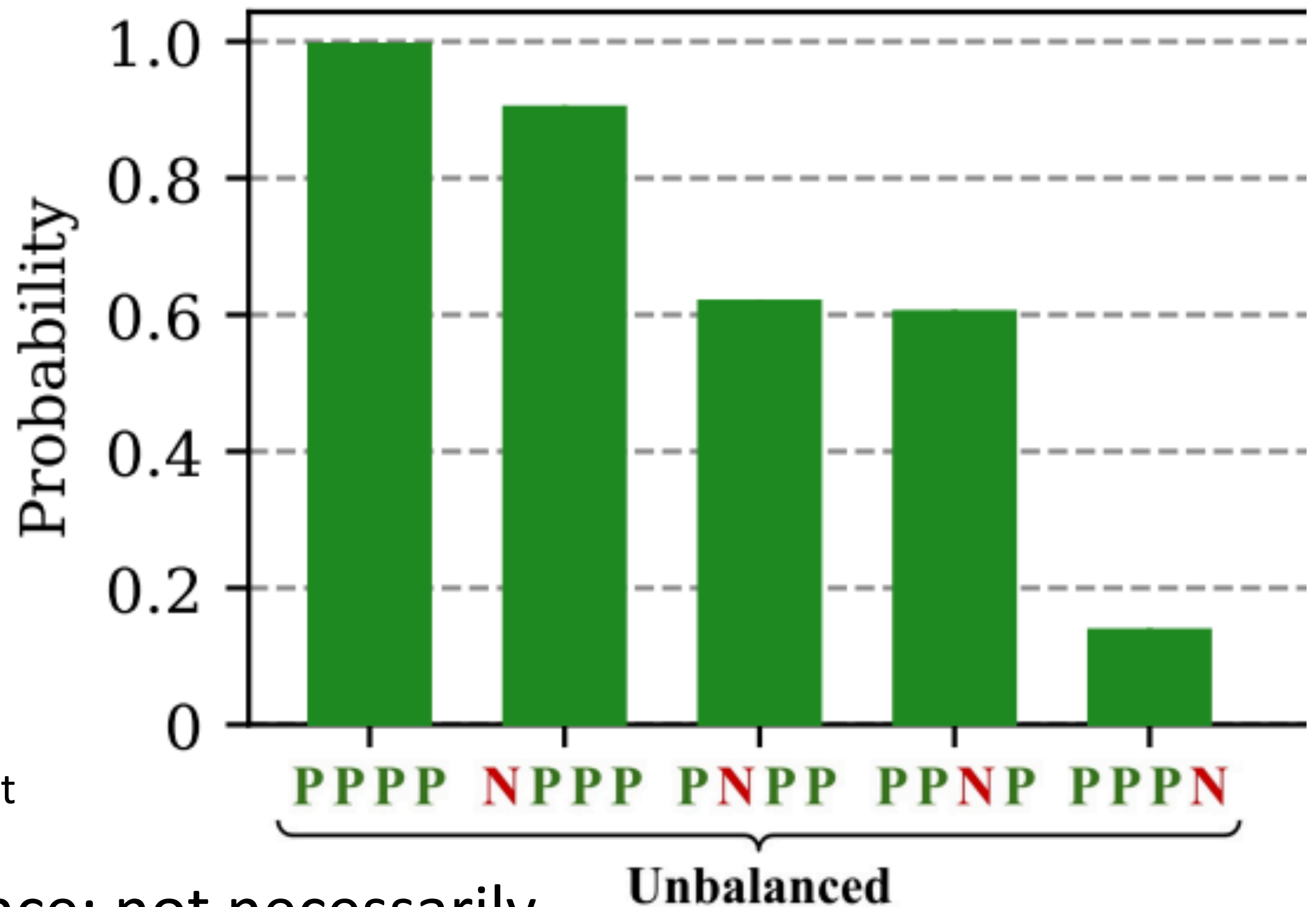
- ▶ Varies even across permutations of training examples
- ▶ x-axis: different collections of train examples.
y-axis: sentiment accuracy. Boxes represent results over different permutations of the data





What can go wrong?

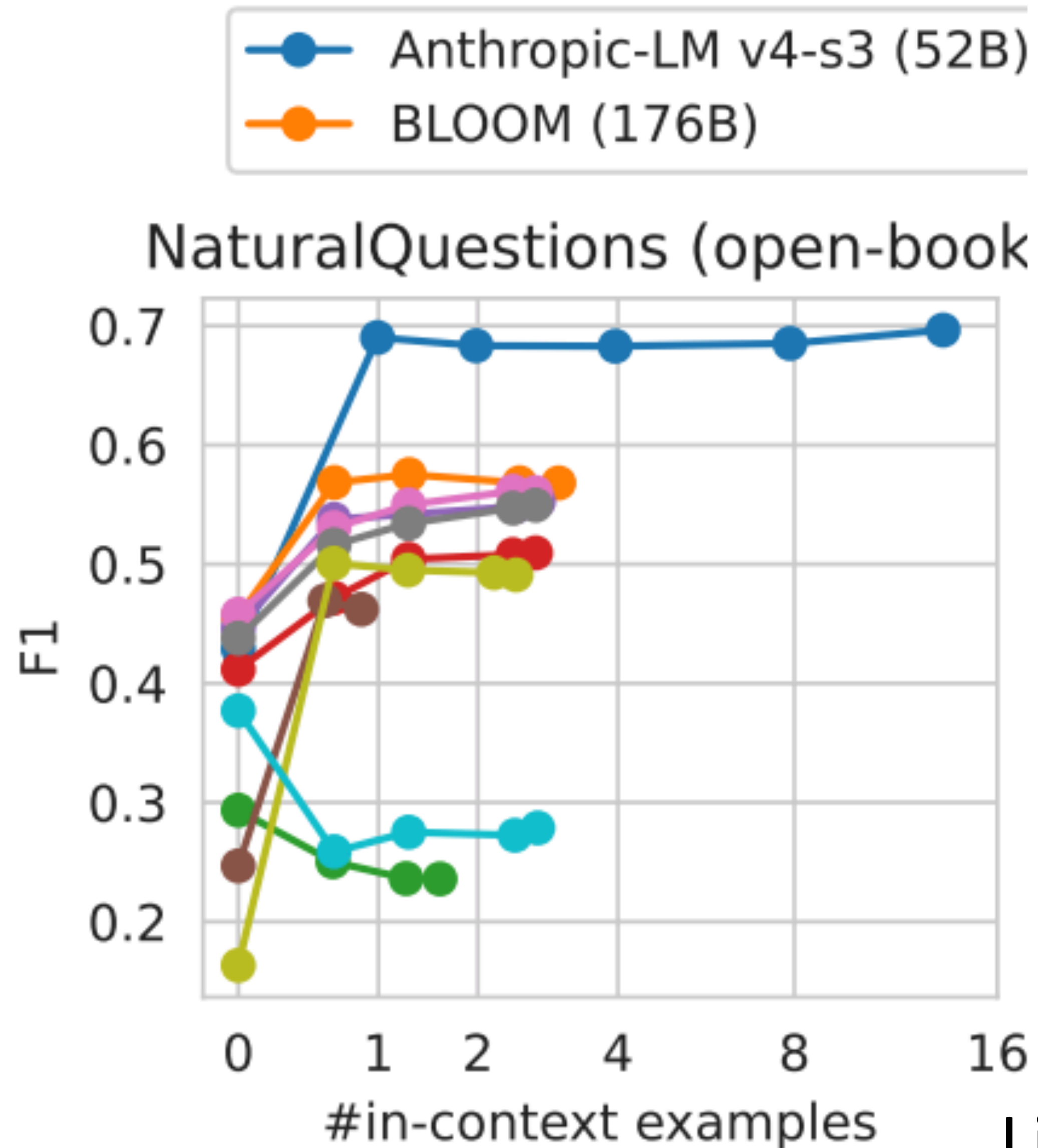
- ▶ Having unbalanced training sets leads to high “default” probabilities of positive; that is, if we feed in a null $\mathbf{x}_{\text{test}}$
- ▶ Solution: “calibrate” the model by normalizing by that probability of null $\mathbf{x}_{\text{test}}$
- ▶ Leads to higher performance; not necessarily crucial with prompt-tuned models





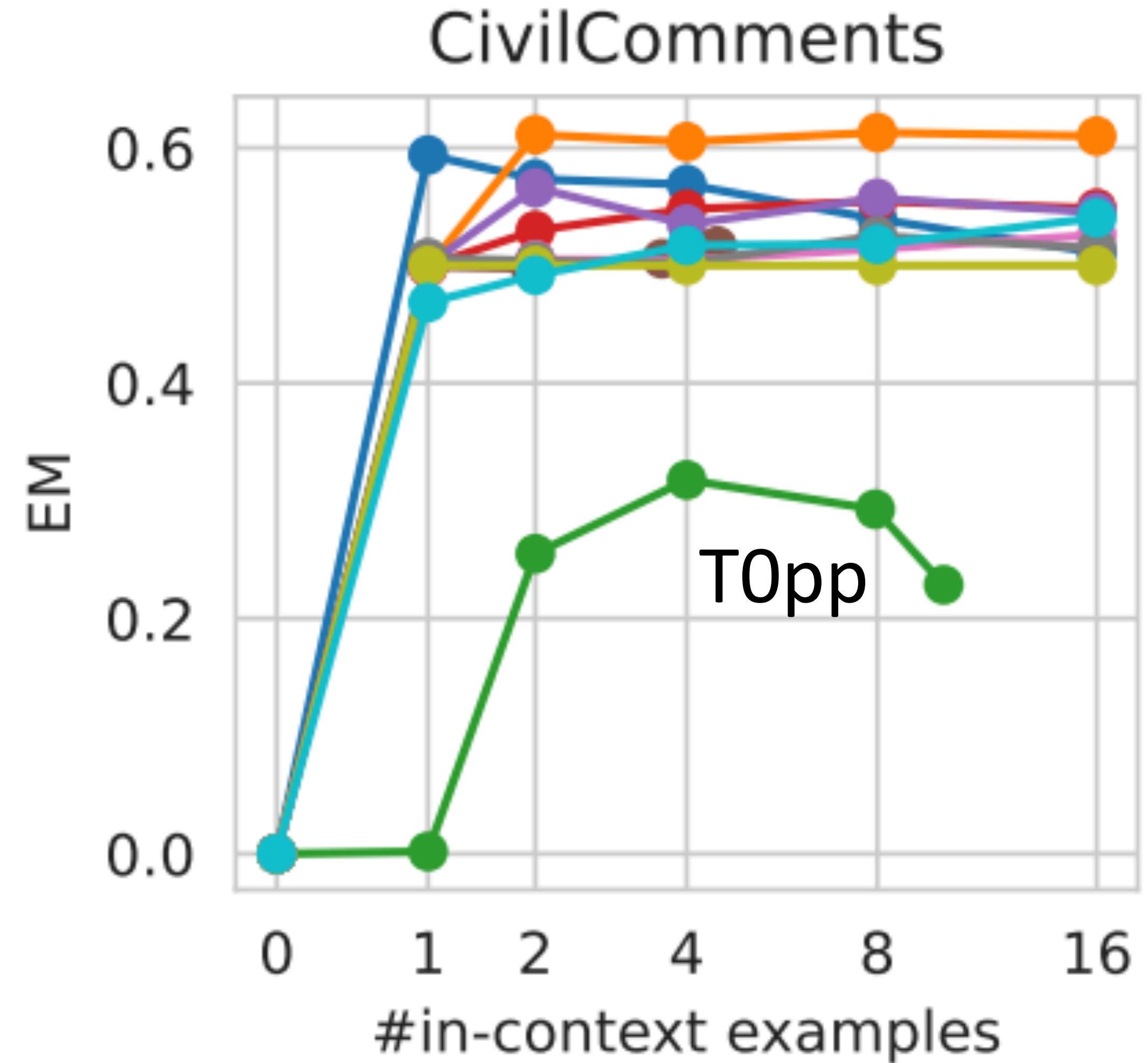
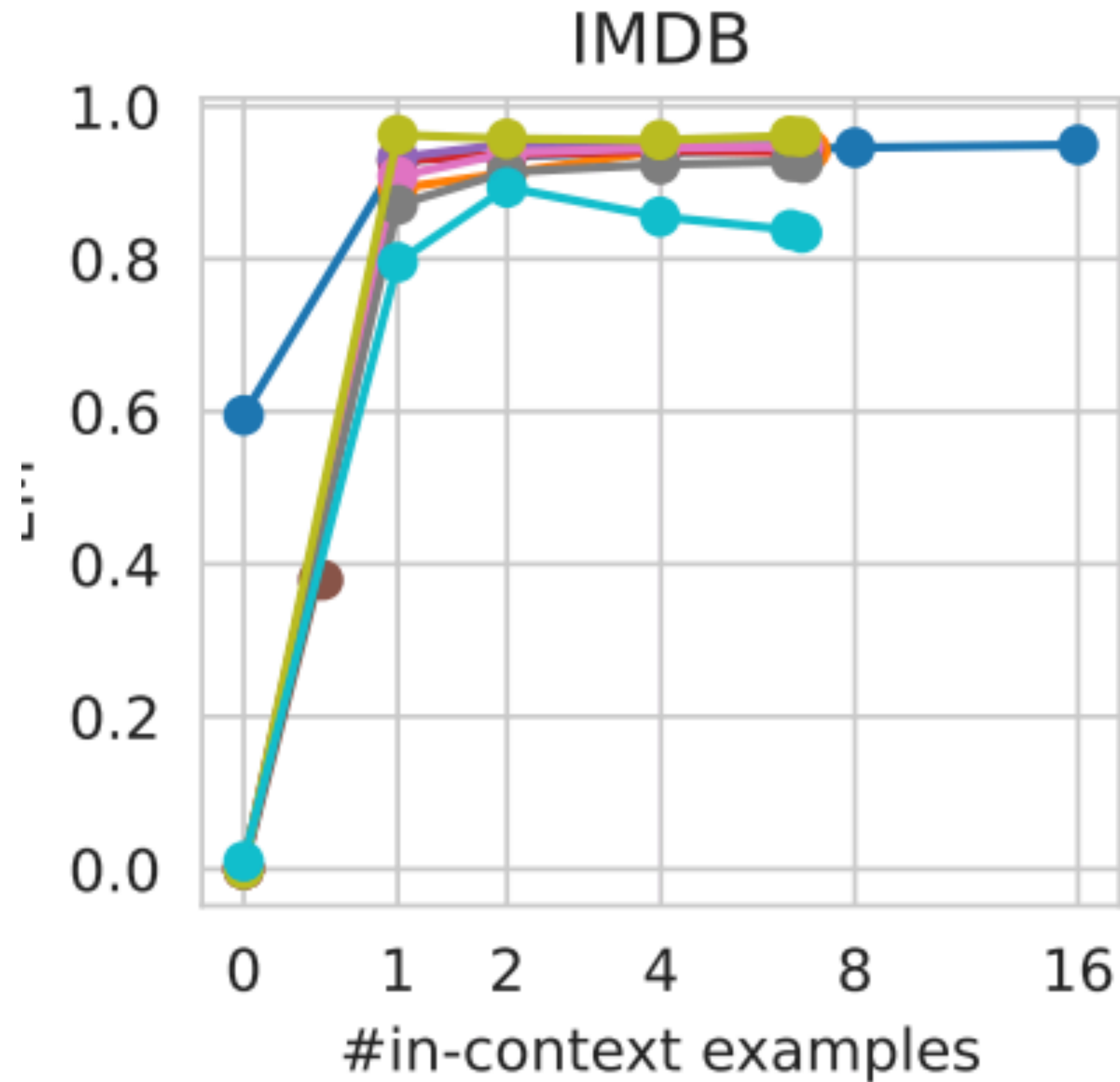
Results: HELM

- ▶ So, how much better is few-shot compared to zero-shot?
- ▶ Each line is a different LM
- ▶ More in-context examples generally leads to better performance
- ▶ What do we see here?





Results: HELM



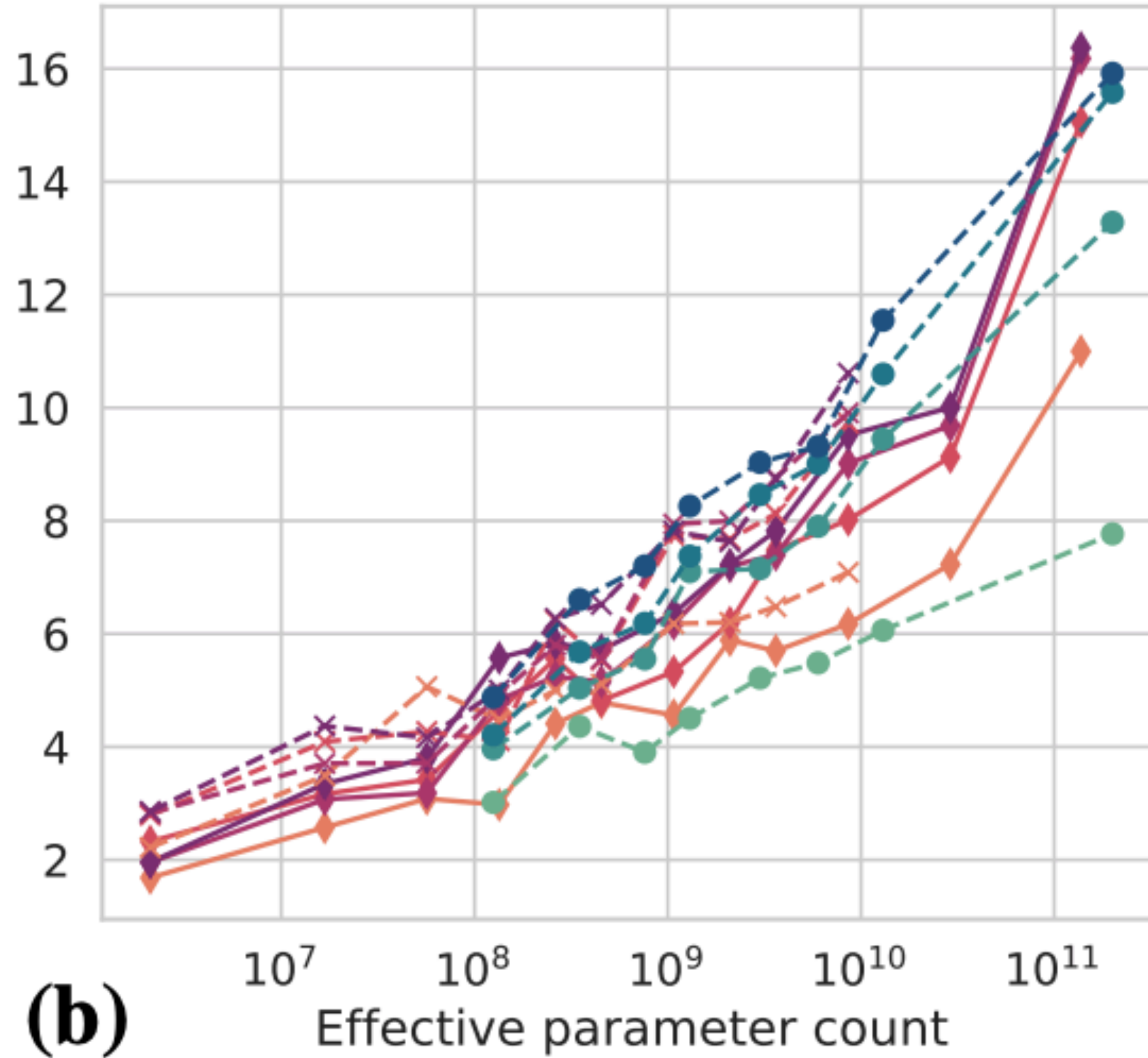
- What trends do these show?



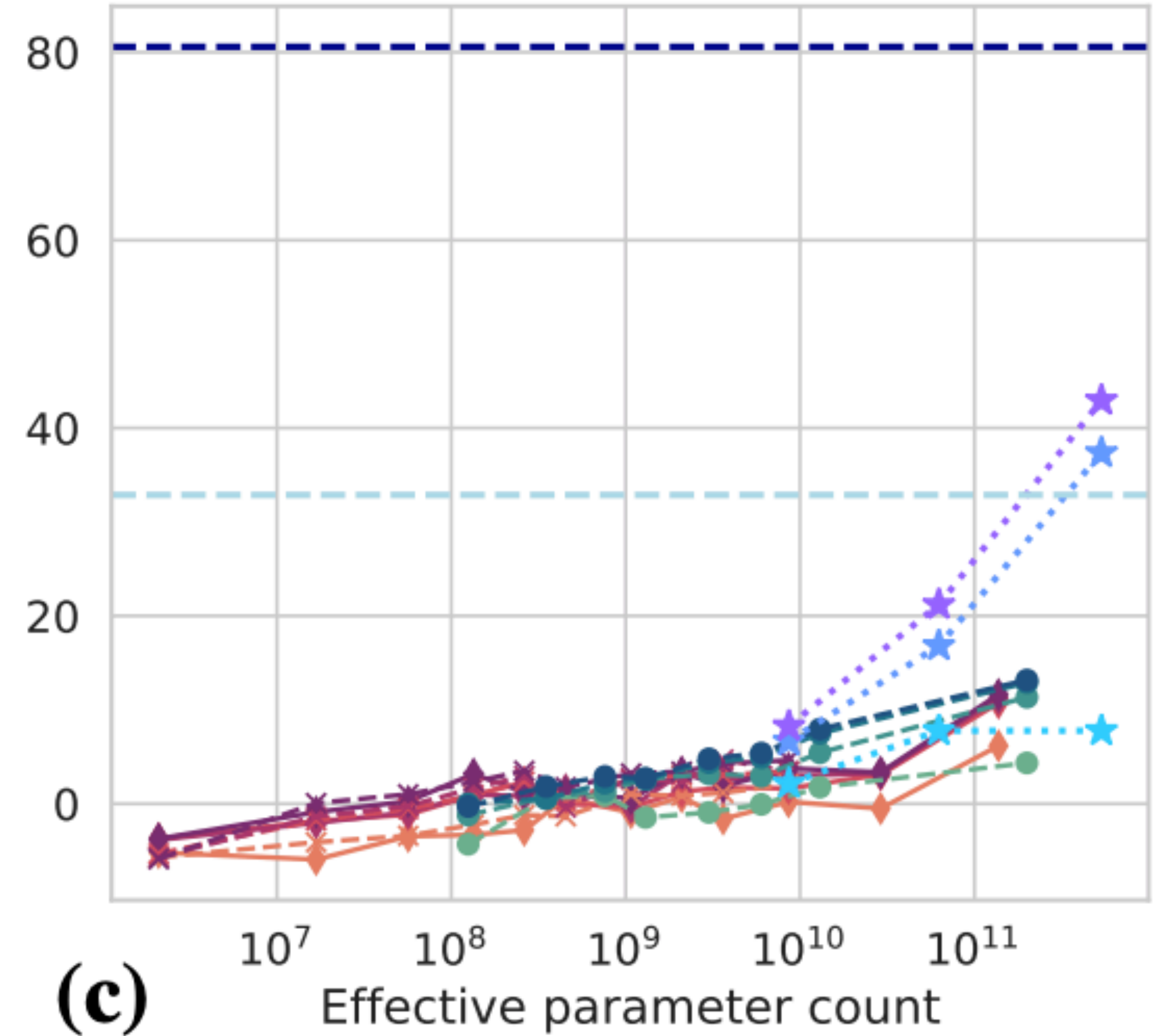
Results: BIG-bench

BIG-G: internal Google model.

Performance on JSON tasks



Performance on BIG-bench Lite



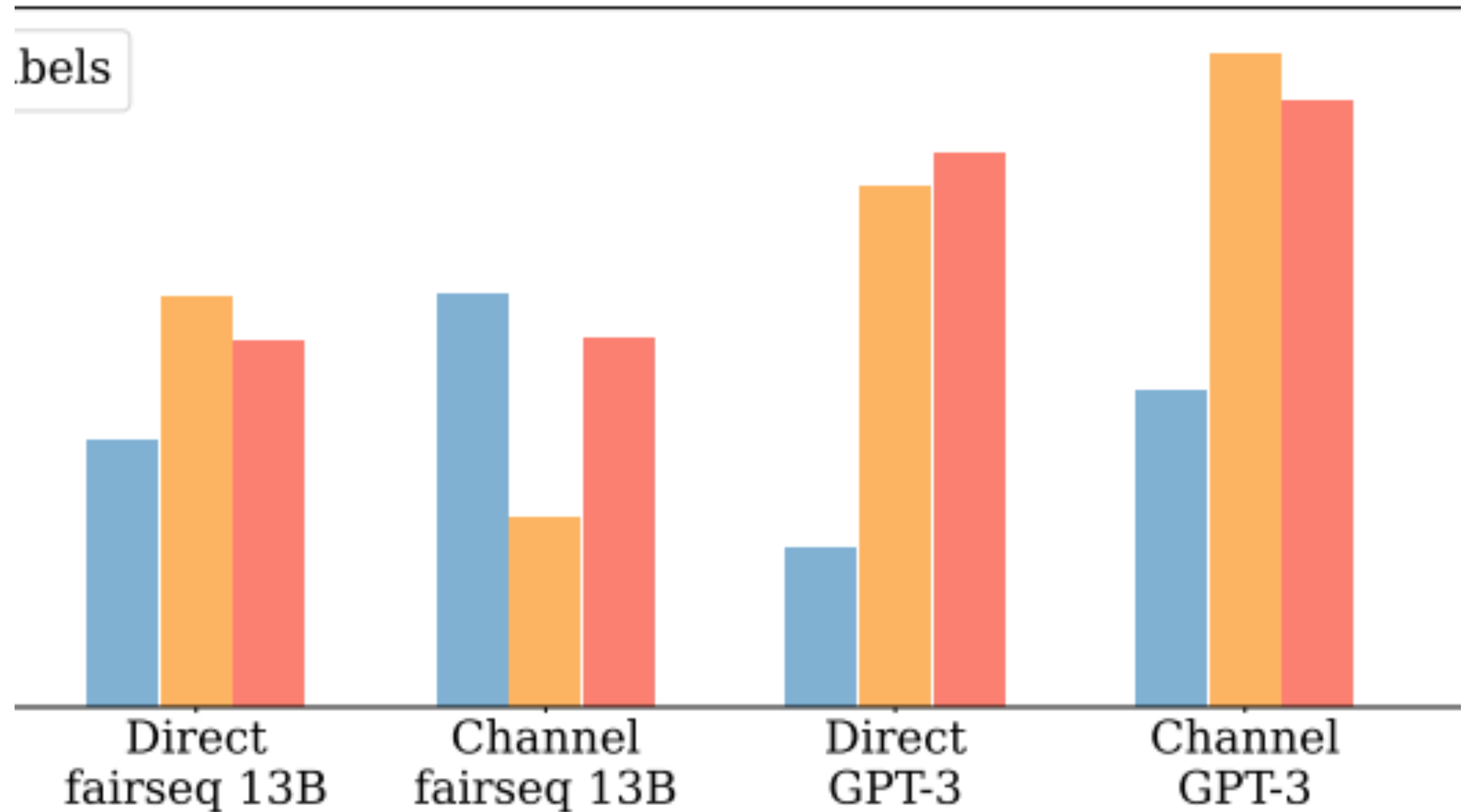
Srivastava et al. (2022)



Rethinking Demonstrations

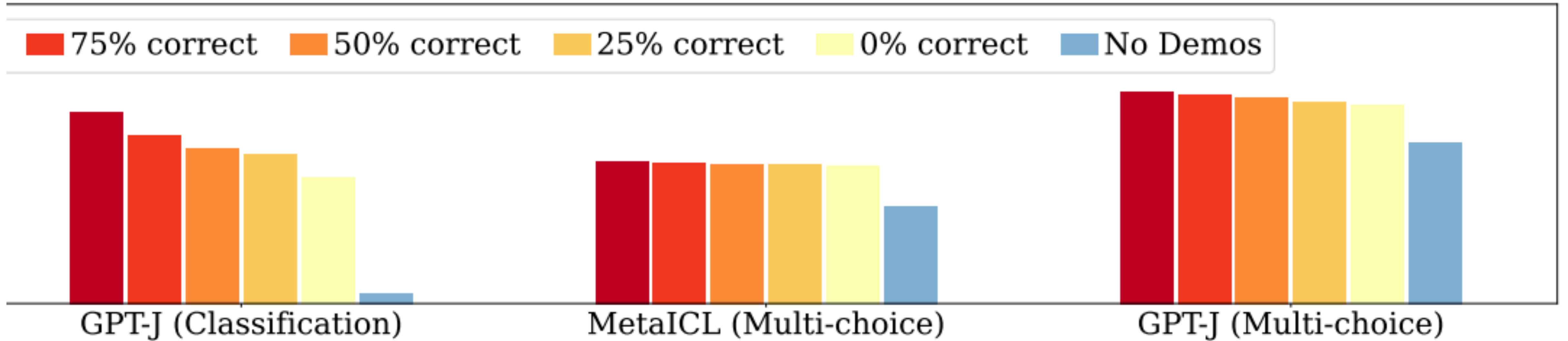
■ No Demos ■ Demos w/ gold labels ■ Demos w/ random labels

- ▶ Surprising result: how necessary even are the demonstrations?
- ▶ Using random labels does not substantially decrease performance??





Rethinking Demonstrations



- ▶ Having even mislabeled demonstrations is much better than having no demonstrations, indicating that the form of the demonstrations is partially responsible for in-context learning



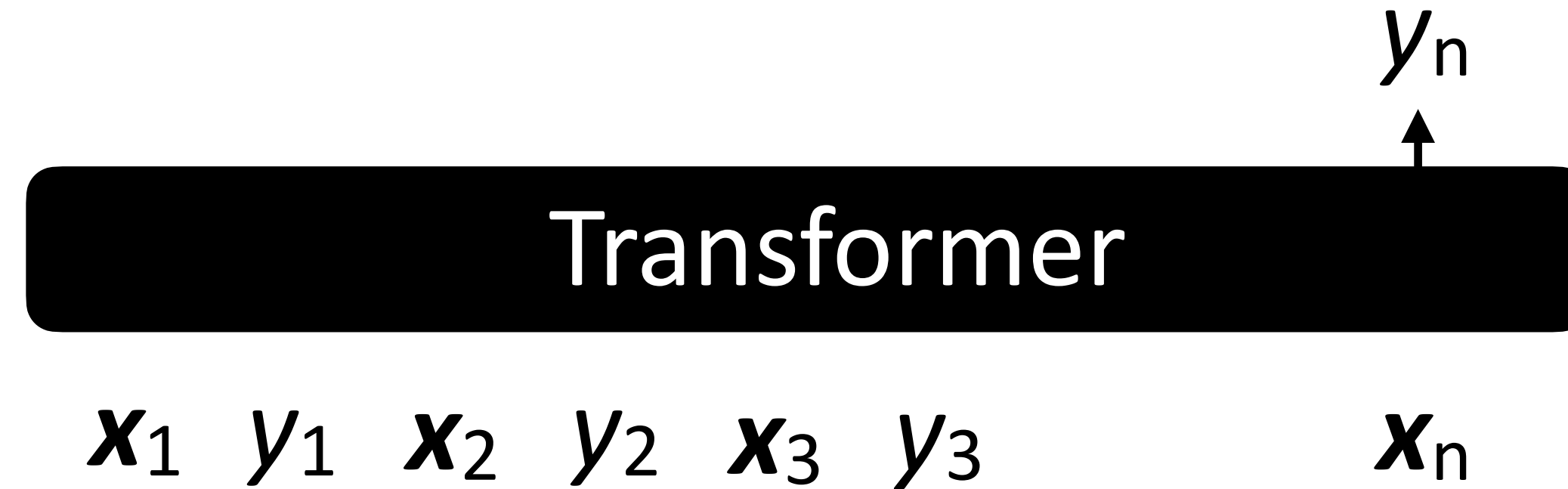
ICL Now

- ▶ Do we still need ICL?
- ▶ Depends on the goal...
 - ▶ Learning format? Generally no (instruction tuning)
 - ▶ Learning task? It depends (how similar to pretraining)

Understanding ICL: Regression



Linear Regression



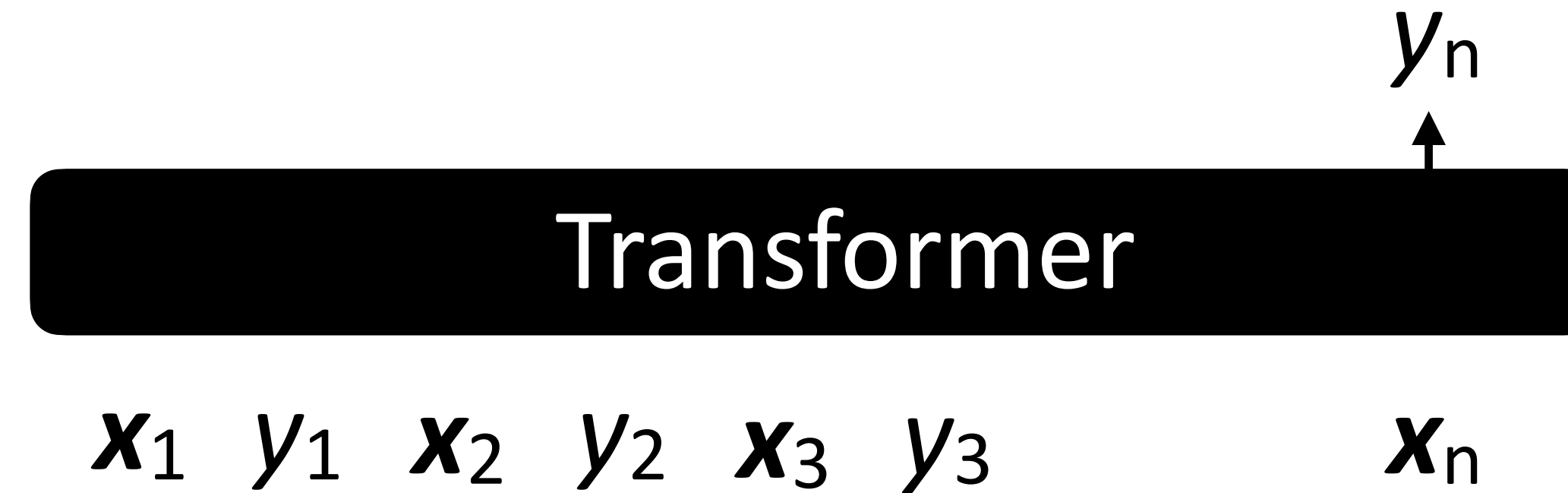
- ▶ Input space is of the form $[y, \mathbf{x}]$, with the “unused” components set to 0
- ▶ See if we can learn regression: given $(\mathbf{x}, y)$ pairs, learn a linear predictor $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$. That is, ground truth is a linear function (synthetic task)
- ▶ Equivalent to minimizing the following loss:

$$\sum_i \mathcal{L}(\mathbf{w}^\top \mathbf{x}_i, y_i) + \lambda \|\mathbf{w}\|_2^2$$

$$\text{minimized by: } \mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top y$$



Linear Regression



- ▶ Question 1: can a Transformer learn to do linear regression?
 - ▶ If we train it to do this task on many examples, does it successfully learn to do “ICL” linear regression on new instances?
 - ▶ If so, there are several different “algorithms” it might correspond to!
- ▶ Question 2: can we inspect what algorithm actually gets implemented?



Linear Regression

- ▶ Most of these proofs (and other papers in this space) rely on Transformers being able to perform several kinds of operations

mov($H; s, t, i, j, i', j'$): selects the entries of the s^{th} column of H between rows i and j , and copies them into the t^{th} column ($t \geq s$) of H between rows i' and j' , yielding the matrix:

$$\left[\begin{array}{c|cc} & H_{:,i-1,t} & \\ H_{:,t} & H_{i':j',s} & H_{:,t+1:} \\ & H_{j,t} & \end{array} \right] .$$

- ▶ How can this be implemented?
What does the attention need to do?



Linear Regression

mov($H; s, t, i, j, i', j'$): selects the entries of the s^{th} column of H between rows i and j , and copies them into the t^{th} column ($t \geq s$) of H between rows i' and j' , yielding the matrix:

$$\left[\begin{array}{c|cc} & H_{:,i-1,t} & \\ H_{:,t} & H_{i':j',s} & H_{:,t+1:} \\ & H_{j,t} & \end{array} \right].$$

mul($H; a, b, c, (i, j), (i', j'), (i'', j'')$): in *each* column $\mathbf{h}$ of H , interprets the entries between i and j as an $a \times b$ matrix A_1 , and the entries between i' and j' as a $b \times c$ matrix A_2 , multiplies these matrices together, and stores the result between rows i'' and j'' , yielding a matrix in which each column has the form $[\mathbf{h}_{:i''-1}, A_1 A_2, \mathbf{h}_{j'':}]^\top$.

- Several more operations as well



Linear Regression

Theorem 1. *A transformer can compute Eq. (11) (i.e. the prediction resulting from single step of gradient descent on an in-context example) with constant number of layers and $O(d)$ hidden space, where d is the problem dimension of the input x . Specifically, there exist transformer parameters θ such that, given an input matrix of the form:*

$$H^{(0)} = \begin{bmatrix} \cdots & 0 & y_i & 0 & \cdots \\ & \mathbf{x}_i & 0 & \mathbf{x}_n & \end{bmatrix}, \quad (12)$$

the transformer's output matrix $H^{(L)}$ contains an entry equal to $\mathbf{w}'^\top \mathbf{x}_n$ (Eq. (11)) at the column index where $\mathbf{x}_n$ is input.

- ▶ Also another update possible based on rank-one updates (Sherman-Morrison)



Proof of Theorem

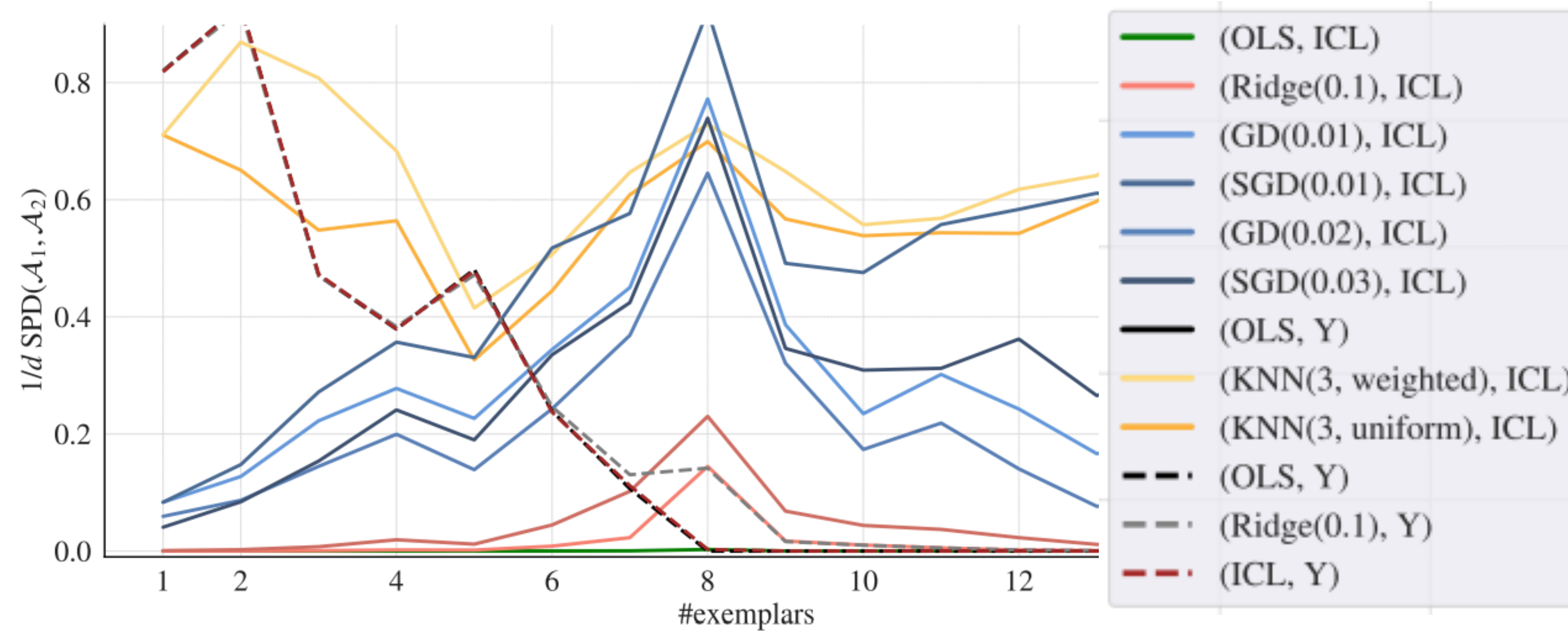
The operations for 1-step SGD with single exemplar can be expressed as following chain (please see proofs for the Transformer implementation of these operations (Lemma 1) in Appendix C):

- $\text{mov}(\cdot; 1, 0, (1, 1 + d), (1, 1 + d))$ (move $\mathbf{x}$)
- $\text{aff}(\cdot; (1, 1 + d), (), (1 + d, 2 + d), W_1 = \mathbf{w})$ ($\mathbf{w}^\top \mathbf{x}$)
- $\text{aff}(\cdot; (1 + d, 2 + d), (0, 1), (2 + d, 3 + d), W_1 = I, W_2 = -I)$ ($\mathbf{w}^\top \mathbf{x} - y$)
- $\text{mul}(\cdot; d, 1, 1, (1, 1 + d), (2 + d, 3 + d), (3 + d, 3 + 2d))$ ($\mathbf{x}(\mathbf{w}^\top \mathbf{x} - y)$)
- $\text{aff}(\cdot; (), (), (3 + 2d, 3 + 3d), b = \mathbf{w},)$ (write $\mathbf{w}$)
- $\text{aff}(\cdot; (3 + d, 3 + 2d), (3 + 2d, 3 + 3d), (3 + 3d, 3 + 4d), W_1 = I, W_2 = -\lambda)$ ($\mathbf{x}(\mathbf{w}^\top \mathbf{x} - y) - \lambda \mathbf{w}$)
- $\text{aff}(\cdot; (3 + 2d, 3 + 3d), (3 + 3d, 3 + 4d), (3 + 2d, 3 + 3d), W_1 = I, W_2 = -2\alpha,)$ ($\mathbf{w}'$)
- $\text{mov}(\cdot; 2, 1, (3 + 2d, 3 + 3d), (3 + 2d, 3 + 3d))$ (move $\mathbf{w}'$)
- $\text{mul}(\cdot; 1, d, 1, (3 + 2d, 3 + 3d), (1, 1 + d), (3 + 3d, 4 + 3d))$ ($\mathbf{w}'^\top \mathbf{x}_2$)



Linear Regression

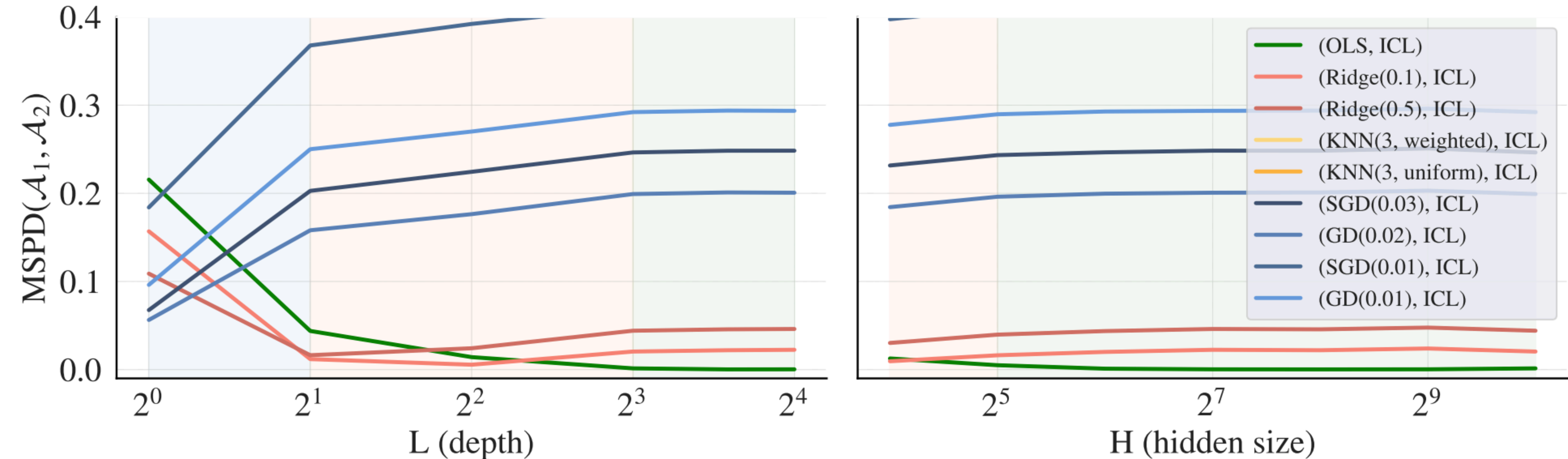
- ▶ Squared prediction difference: L2 between different predictors
- ▶ When no noise: ICL matches ordinary least square (OLS) almost exactly





Linear Regression

- Squared prediction difference: L2 between different predictors



- What gets learned changes with depth. Low-depth: more like GD. Medium-depth: more like ridge. High-depth: OLS



ICL = Gradient Descent?

- ▶ One hypothesis: ICL is implicitly performing gradient descent
- ▶ Hypothesis: for any set of transformer weights you could have gotten via training, and any well-defined task, you can provide a set of examples that would result in ICL being algorithmically the same as gradient descent
- ▶ Is this true? Prior work including Akyürek has strong assumptions.
- ▶ These assumptions are stronger than what is true in practice.
 - ▶ Prior work trains *with* ICL objective (LLMs don't)
 - ▶ Tasks are restricted



Takeaways

- ▶ Zero- and few-shot prompting are very powerful ways of specifying new tasks at inference time
- ▶ For zero-shot: form of the prompt matters, we'll see more example next times when we look at chain-of-thought
- ▶ For few-shot: number and order of the examples matters, prompt matters a bit less
- ▶ Several analyses of why it works: it can learn to do regression and we know a bit about mechanisms that may be responsible for it